Phase 1: Framework & Engine Initialization

📌 Phase 1 Brief

Before we implement specific data structures, we need to build the underlying engine of our data visualization application. Phase 1 is strictly about setting up the MVC architecture, the graphics window, the UI control panel, and the timeline logic.

By the end of Phase 1, the framework must be able to:

1. Open a cross-platform graphics window.
2. Provide a UI for users to customize the color, size, and style of the visualization.
3. Support “Run-step-by-step” controls (next step, previous step, final step).
4. Support “Run-at-once” with an adjustable visualizing speed.
5. Provide a UI space to highlight the corresponding source codes when visualizing operations.
6. Read initial data manually from the keyboard, from an external file, or via randomized data.

To work efficiently and avoid Git conflicts, Phase 1 tasks are strictly divided into 4 domains.

🛠️ GitHub Issues to Create

Instructions: Copy everything below each title and paste it into a New Issue on GitHub. Assign one team member to each issue.

Issue 1: [Core Architect] Implement Timeline State and Playback Logic

Assignee: [Name of Member 1] **Description:** Build the Model and Controller logic that manages the application’s state and animation timeline. This system needs to act as a “movie projector” that plays back frames of an operation.

Tasks:

- [ ] Define the `Frame` struct (in `.h` and `.cpp` files) to hold the visual state and current code line ID of a single step.
- [ ] Define the `Timeline` struct to hold a sequence of `Frame` objects.
- [ ] Implement the `Playback` controller to handle “Run-step-by-step” logic (go to next step, previous step, final step).
- [ ] Implement “Run-at-once” logic that continuously advances the frame from the first step to the final step based on a timer.
- [ ] Connect the `AppEngine` loop so it updates the state continuously.

Issue 2: [Graphics Engineer] Implement Base Rendering System

Assignee: [Name of Member 2] **Description:** Initialize the graphics library (e.g., SFML/Raylib) and write the generic drawing functions that Phase 2 data structures will use to draw themselves on the screen. Do not write data structure logic here.

Tasks:

- [ ] Set up the `Window` struct to initialize the main application window and handle the main render loop.
- [ ] Define the `Theme` struct (in `.h` and `.cpp` files) to store customizable variables for color, size, and style.
- [ ] Implement generic shape drawing functions in `Renderer.cpp` (e.g., `drawCircle`, `drawRectangle`, `drawLine`).
- [ ] Implement text rendering functions in `Renderer.cpp` to display data values inside shapes.
- [ ] Ensure all drawing functions dynamically read from the `Theme` struct to apply user preferences.

Issue 3: [UI/UX Developer] Build Control Panel and Code Highlighter

Assignee: [Name of Member 3] **Description:** Implement the user interface (using Dear ImGui or native UI) that wraps around the visualization window.

Tasks:

- [ ] Set up the `UIManager` struct and initialize the UI library within the main application window.
- [ ] Build the persistent playback control buttons: Play, Pause, Step Forward, and Step Backward.
- [ ] Create a UI slider to allow the user to change the visualizing speed.
- [ ] Create input forms to allow users to add new data, delete data, update data, and search for specific data points.
- [ ] Build a dedicated UI text panel to display and highlight the corresponding source codes when visualizing operations.

Issue 4: [Data Manager] Implement File I/O and Enforce Git Standards

Assignee: [Name of Member 4] **Description:** Handle all data ingestion mechanics so Phase 2 structures can easily be populated. Also, act as the Git reviewer to ensure we meet the strict version control requirements.

Tasks:

- [ ] Write a generic file parser in the `Controller` that reads an external text/CSV file and returns a tandard array/vector of strings or integers.
- [ ] Implement logic to generate randomized data arrays.
- [ ] Implement logic to capture manual keyboard entry from the UI and convert it into data arrays.
- [ ] Ensure empty data structures are safely handled if no input data is provided.
- [ ] Enforce Git rules: Review PRs, ensure the commit list is growing, and verify every single `struct` as exactly 2 files (`.h` and `.cpp`).